핵심 API 및 함수 분석

개요

파일코인 proof 시스템의 핵심 API는 `filecoin-proofs` 크레이트에서 제공되며, 주로 sealing과 PoSt 관련 기능을 포함합니다. 이 문서에서는 주요 API 함수들의 매개변수, 반환값, 그리고 내부 동작 과정을 상세히 분석합니다.

Sealing API 함수들

seal_pre_commit_phase1

함수 시그니처:

```
pub fn seal_pre_commit_phase1<R, S, T, Tree: 'static + MerkleTreeTrait>(
porep_config: &PoRepConfig,
cache_path: R,
in_path: S,
out_path: T,
prover_id: ProverId,
sector_id: SectorId,
ticket: Ticket,
piece_infos: &[PieceInfo],
) -> Result<SealPreCommitPhase1Output<Tree>>
```

매개변수 설명:

- `porep_config` : PoRep 설정 정보 (섹터 크기, 파티션 수 등)
- `cache_path` : 캐시 디렉터리 경로 (머클 트리와 레이블 저장)
- `in_path` : 입력 데이터 파일 경로 (unsealed 데이터)
- `out_path` : 출력 파일 경로 (replica 데이터가 저장될 위치)
- `prover_id` : 증명자 식별자 (32바이트)
- `sector_id` : 섹터 식별자
- `ticket` : 랜덤성 소스 (32바이트)
- `piece_infos` : 피스 정보 배열 (commitment와 크기)

반환값:

```
pub struct SealPreCommitPhase1Output<Tree: MerkleTreeTrait> {
pub labels: Labels<Tree>, // 생성된 레이블들
pub config: StoreConfig, // 데이터 트리 설정
pub comm_d: Commitment, // 데이터 commitment
}
```

주요 동작 과정:

1. **입력 검증**: 파일 경로와 크기 검증
2. **데이터 복사**: unsealed 데이터를 replica 위치로 복사
3. **CommD 생성**: 입력 데이터로부터 머클 트리 구축 및 commitment 계산
4. **피스 검증**: 제공된 피스 정보와 CommD 일치성 확인
5. **Replica ID 생성**: prover_id, sector_id, ticket, comm_d로부터 계산
6. **레이블 생성**: Stacked DRG의 첫 번째 단계 실행

seal_pre_commit_phase2

함수 시그니처:

```
pub fn seal_pre_commit_phase2<R, S, Tree: 'static + MerkleTreeTrait>(
porep_config: &PoRepConfig,
phase1_output: SealPreCommitPhase1Output<Tree>,
cache_path: S,
replica_path: R,
) -> Result<SealPreCommitOutput>
```

매개변수 설명:

- `porep_config` : PoRep 설정 정보
- `phase1_output` : Phase 1의 출력 결과
- `cache_path` : 캐시 디렉터리 경로
- `replica_path` : replica 파일 경로

반환값:

```
pub struct SealPreCommitOutput {
pub comm_r: Commitment, // Replica commitment
pub comm_d: Commitment, // Data commitment
}
```

주요 동작 과정:

1. **캐시 업데이트**: 레이블과 설정 경로 업데이트

2. **데이터 로딩**: replica 파일을 메모리 맵으로 로딩
3. **데이터 트리 복원**: 디스크에서 CommD 트리 로딩
4. **인코딩 실행**: Stacked DRG의 두 번째 단계 (레이어별 인코딩)
5. **머클 트리 구축**: TreeC와 TreeRLast 생성
6. **CommR 계산**: 최종 replica commitment 생성
7. **보조 데이터 저장**: p_aux와 t_aux 캐시에 저장

seal_commit_phase1

함수 시그니처:

```
pub fn seal_commit_phase1<T: AsRef<Path>, Tree: 'static + MerkleTreeTrait>(
porep_config: &PoRepConfig,
cache_path: T,
replica_path: T,
prover_id: ProverId,
sector_id: SectorId,
ticket: Ticket,
seed: Ticket,
pre_commit: SealPreCommitOutput,
piece_infos: &[PieceInfo],
) -> Result<SealCommitPhase1Output<Tree>>
```

매개변수 설명:

- `seed` : PoRep challenge 생성을 위한 시드 (32바이트)

- `pre_commit` : PreCommit 단계의 출력 결과
- 기타 매개변수는 이전 함수들과 동일

반환값:

```
pub struct SealCommitPhase1Output<Tree: MerkleTreeTrait> {
pub vanilla_proofs: Vec<Vec<VanillaSealProof<Tree>>>, // 바닐라 증명들
pub comm_r: Commitment, // Replica commitment
pub comm_d: Commitment, // Data commitment
pub replica_id: <Tree::Hasher as Hasher>::Domain, // Replica ID
pub seed: Ticket, // Challenge 시드
pub ticket: Ticket, // 원본 티켓
}
```

주요 동작 과정:

1. **입력 검증**: commitment와 피스 정보 검증
2. **보조 데이터 로딩**: p_aux와 t_aux 캐시에서 로딩
3. **캐시 복원**: TemporaryAuxCache 인스턴스 생성
4. **Public Inputs 구성**: replica_id, tau, seed 설정
5. **Private Inputs 구성**: p_aux와 t_aux_cache 설정
6. **바닐라 증명 생성**: 모든 파티션에 대한 증명 생성
7. **검증**: 생성된 증명의 유효성 확인

seal_commit_phase2

함수 시그니처:

```
pub fn seal_commit_phase2<Tree: 'static + MerkleTreeTrait>(
porep_config: &PoRepConfig,
phase1_output: SealCommitPhase1Output<Tree>,
prover_id: ProverId,
sector_id: SectorId,
) -> Result<SealCommitOutput>
```

매개변수 설명:

- `phase1_output` : Phase 1의 출력 결과
- 기타 매개변수는 이전 함수들과 동일

반환값:

```
pub struct SealCommitOutput {
pub proof: Vec<u8>, // 최종 SNARK 증명 바이트
}
```

주요 동작 과정:

1. **입력 검증**: commitment와 시드 유효성 확인
2. **Public Inputs 구성**: replica_id, tau, seed 설정
3. **Groth16 매개변수 로딩**: 회로 증명을 위한 매개변수
4. **회로 증명 생성**: 바닐라 증명을 SNARK 증명으로 변환
5. **증명 직렬화**: MultiProof를 바이트 배열로 변환
6. **집계 처리**: NonInteractivePoRep인 경우 집계 증명 생성

7. **검증**: 최종 증명의 유효성 확인

PoSt API 함수들

generate_winning_post

함수 시그니처:

```
pub fn generate_winning_post<Tree: 'static + MerkleTreeTrait>(
post_config: &PoStConfig,
randomness: &ChallengeSeed,
replicas: &[(SectorId, PrivateReplicaInfo<Tree>)],
prover_id: ProverId,
) -> Result<SnarkProof>
```

매개변수 설명:

- `post_config` : PoSt 설정 정보 (타입, 섹터 수, challenge 수)
- `randomness` : challenge 생성을 위한 랜덤성 (32바이트)
- `replicas` : 증명할 replica 정보 배열
- `prover_id` : 증명자 식별자

반환값:

- `SnarkProof` : SNARK 증명 바이트 배열

주요 동작 과정:

1. **설정 검증**: PoSt 타입이 Winning인지 확인
2. **replica 수 검증**: 설정된 섹터 수와 일치하는지 확인

3. **안전한 commitment 변환**: randomness와 prover_id 변환

4. **머클 트리 로딩**: 각 replica의 머클 트리 구성

5. **Public/Private Inputs 구성**:

- Public: randomness, prover_id, sector commitments
- Private: 머클 트리, comm_c, comm_r_last

6. **증명 생성**: FallbackPoStCompound를 통한 SNARK 증명

7. **바이트 변환**: 증명을 바이트 배열로 직렬화

generate_window_post

함수 시그니처:

```
pub fn generate_window_post<Tree: 'static + MerkleTreeTrait>(
post_config: &PoStConfig,
randomness: &ChallengeSeed,
replicas: &BTreeMap<SectorId, PrivateReplicaInfo<Tree>>,
prover_id: ProverId,
) -> Result<SnarkProof>
```

매개변수 설명:

- `replicas` : BTreeMap 형태의 replica 정보 (정렬된 섹터 ID)
- 기타 매개변수는 generate_winning_post와 동일

주요 동작 과정:

1. **설정 검증**: PoSt 타입이 Window인지 확인

2. **파티션 계산**: replica 수에 따른 파티션 수 결정

3. **병렬 트리 로딩**: 여러 replica의 머클 트리를 병렬로 구성

4. **Public/Private Inputs 구성**: Winning PoSt와 유사하지만 파티션 고려

5. **증명 생성**: 파티션별로 분할된 증명 생성

6. **증명 집계**: 여러 파티션의 증명을 하나로 결합

데이터 복원 API

unseal_range

함수 시그니처:

```
pub fn unseal_range<P, R, W, Tree>(
porep_config: &PoRepConfig,
cache_path: P,
mut sealed_sector: R,
unsealed_output: W,
prover_id: ProverId,
sector_id: SectorId,
comm_d: Commitment,
ticket: Ticket,
offset: UnpaddedByteIndex,
num_bytes: UnpaddedBytesAmount,
) -> Result<UnpaddedBytesAmount>
```

매개변수 설명:

- `sealed_sector` : 봉인된 섹터 데이터 읽기 소스
- `unsealed_output` : 복원된 데이터 쓰기 대상
- `offset` : 복원할 데이터의 시작 위치 (unpadded 바이트 인덱스)
- `num_bytes` : 복원할 바이트 수 (unpadded)
- 기타 매개변수는 sealing 함수들과 유사

반환값:

- `UnpaddedBytesAmount` : 실제로 복원된 바이트 수

주요 동작 과정:

1. **입력 검증**: comm_d가 유효한지 확인
2. **Replica ID 재생성**: 원본 sealing 매개변수로부터 계산
3. **봉인된 데이터 로딩**: 전체 섹터 데이터를 메모리로 로딩
4. **역변환 실행**:
   - Stacked DRG의 레이어별 역변환
   - 각 레이어의 인코딩을 순차적으로 해제
5. **범위 추출**: 요청된 offset과 num_bytes에 해당하는 데이터 추출
6. **패딩 제거**: Fr32 패딩을 제거하여 원본 데이터 복원
7. **출력 쓰기**: 복원된 데이터를 출력 스트림에 쓰기

get_unsealed_range

함수 시그니처:

```
pub fn get_unsealed_range<T: Into<PathBuf> + AsRef<Path>, Tree: 'static + Merk
porep_config: &PoRepConfig,
cache_path: T,
```

```
sealed_path: T,
output_path: T,
prover_id: ProverId,
sector_id: SectorId,
comm_d: Commitment,
ticket: Ticket,
offset: UnpaddedByteIndex,
num_bytes: UnpaddedBytesAmount,
) -> Result<UnpaddedBytesAmount>
```

매개변수 설명:

- `sealed_path` : 봉인된 섹터 파일 경로
- `output_path` : 복원된 데이터를 저장할 파일 경로
- 기타 매개변수는 unseal_range와 동일

주요 동작 과정:

1. **출력 파일 생성**: output_path에 새 파일 생성
2. **버퍼 래퍼 생성**: BufWriter로 출력 성능 최적화
3. **메모리 맵 복원**: unseal_range_mapped 호출
4. **파일 시스템 동기화**: 복원된 데이터를 디스크에 저장

API 사용 패턴

일반적인 Sealing 워크플로우

```
// 1. PreCommit Phase 1
let  phase1_output = seal_pre_commit_phase1(
&porep_config,
cache_path,
unsealed_path,
sealed_path,
prover_id,
sector_id,
ticket,
&piece_infos,
)?;


// 2. PreCommit Phase 2
let  precommit_output = seal_pre_commit_phase2(
&porep_config,
phase1_output,
cache_path,
sealed_path,
)?;


// 3. Commit Phase 1
let  commit_phase1_output = seal_commit_phase1(
```

```
&porep_config,

cache_path,

sealed_path,

prover_id,

sector_id,

ticket,

seed,

precommit_output,

&piece_infos,

)?;


// 4. Commit Phase 2

let  commit_output = seal_commit_phase2(

&porep_config,

commit_phase1_output,

prover_id,

sector_id,

)?;
```

PoSt 증명 생성 워크플로우

```
// Winning PoSt

let  winning_proof = generate_winning_post(
```

```
&post_config,

&randomness,

&replicas,

prover_id,

)?;


// Window PoSt

let  window_proof = generate_window_post(

&post_config,

&randomness,

&replica_map,

prover_id,

)?;
```

데이터 복원 워크플로우

```
// 특정 범위의 데이터 복원

let  restored_bytes = get_unsealed_range::<_, Tree>(

&porep_config,

cache_path,

sealed_path,

output_path,

prover_id,
```

```
sector_id,

comm_d,

ticket,

offset,

num_bytes,

)?;
```

성능 고려사항

메모리 사용량

- **seal_pre_commit_phase1**: 섹터 크기만큼의 메모리 필요 (32GB 섹터의 경우 32GB RAM)
- **seal_pre_commit_phase2**: 머클 트리 구축을 위한 추가 메모리 필요
- **unseal_range**: 전체 섹터를 메모리에 로딩하므로 섹터 크기만큼 필요

디스크 I/O

- **캐시 디렉터리**: 머클 트리와 레이블 저장을 위한 충분한 공간 필요
- **임시 파일**: 중간 계산 결과 저장을 위한 추가 공간
- **순차 접근**: 가능한 한 순차적 디스크 접근 패턴 사용

병렬 처리

- **멀티코어 활용**: Rayon을 통한 병렬 처리 지원
- **GPU 가속**: CUDA 기능 활성화 시 GPU 활용 가능
- **파티션 처리**: 큰 증명을 여러 파티션으로 분할하여 병렬 처리

오류 처리

일반적인 오류 유형

- **파일 시스템 오류**: 경로 접근 불가, 디스크 공간 부족
- **메모리 부족**: 큰 섹터 처리 시 메모리 부족
- **데이터 무결성**: commitment 불일치, 손상된 캐시 파일
- **매개변수 오류**: 잘못된 설정값, 호환되지 않는 매개변수

오류 복구 전략

- **캐시 재생성**: 손상된 캐시 파일 감지 시 자동 재생성
- **부분 재시작**: 실패한 단계부터 재시작 가능
- **검증 단계**: 각 단계마다 결과 검증으로 조기 오류 감지